UNIVERSIDAD POLITECNICA DEL BICENTENARIO



REPORTE DE PRACTICAS

Práctica 2: Control ON-OFF de nivel

Presenta:

Alejandro Almanza Perez

Vanessa Guadalupe Soria Zavala

Adán Rodriguez Ortiz

America Aranzazu Rodriguez Perez

Matricula:

24033177

24030215

24033563

2403

Asignatura:

Sistemas Embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto a 11 de septiembre de 2026

INDICE

Introducción ... 3
Marco Teórico ... 3
Control ON-OFF de temperatura ... 8
PRÁCTICA 3. Sensor de temperatura y foco ... 8
Objetivo ... 8
Material requerido ... 8
Asignación de pines ... 8
Desarrollo ... 9
Diseño del circuito ... 9
Conexiones ... 10
Programación ... 11
Código en Wokwi primera parte del circuito (estructura) ... 11
Funcionamiento ... 20
Resultados ... 20
Observaciones y conclusiones ... 21
Referencias ... 22

Lista de Figuras

Figura 1. Circuito de la Practica Control ON-OFF de temperatura ... 10
Figura 2. Primera parte del circuito en diseñado en Wokwi (estructura) ... 13
Figura 3. Primera parte del circuito en diseñado en Wokwi (en funcionamiento) ... 13
Figura 4. Vista General del Circuito Físico ... 17
Figura 5. Vista General del Circuito Físico en Funcionamiento ... 18
Figura 6. Ventilador Implementado en la Caja ... 18
Figura 7. Datos Obtenidos Durante el Funcionamiento de la Practica ... 19

Introducción

El control de temperatura es fundamental en procesos que requieren condiciones térmicas estables, como incubadoras, cámaras ambientales, hornos y sistemas de climatización. A diferencia de otras variables, la temperatura cambia lentamente, por lo que el comportamiento del sistema depende tanto del control como de las características físicas del entorno.

En esta práctica se implementó un control ON-OFF de temperatura con histéresis usando una tarjeta ESP32 para mantener la temperatura dentro de una caja de cartón. Un sensor digital DS18B20 mide la temperatura y una pantalla LCD I2C muestra la lectura junto con los umbrales LL y HL, que el usuario ajusta con tres botones. Cuando la temperatura baja a LL, se enciende un foco como calefactor, y cuando sube a HL, se activa una celda Peltier con disipador para enfriar. Los dispositivos se conmutan mediante relevadores, y un ventilador mantiene el aire circulando dentro de la caja.

El reporte presenta el marco teórico, el desarrollo de la práctica, los resultados, donde se destaca la lenta respuesta térmica de la caja, y las observaciones y conclusiones.

Marco Teórico

Los sistemas de automatización industrial requieren dispositivos capaces de procesar información y tomar decisiones a partir de distintas condiciones de entrada. Para comprender el funcionamiento de la práctica desarrollada, es necesario abordar primero los conceptos de sistema embebido, microcontrolador, señales digitales, niveles lógicos y operaciones lógicas básicas.

Sistemas embebidos

Un sistema embebido es un sistema de cómputo diseñado para realizar una función específica, cuyo procesamiento central recae en un microcontrolador o microprocesador integrado junto con memoria e interfaces de entrada/salida en un mismo circuito. A diferencia de una computadora de propósito general, un sistema embebido se dedica a una tarea concreta, lo cual le permite ejecutarla con mayor precisión, rapidez y menor consumo de recursos.

Estos sistemas combinan hardware y software especializados: el hardware aporta los componentes físicos (sensores, actuadores, memoria e interfaces de comunicación), mientras que el software embebido gestiona su funcionamiento, procesa datos y controla la interacción con el usuario o con otros dispositivos. Entre

sus principales ventajas se encuentran el control preciso de procesos, la reducción de costos operativos, la posibilidad de un diseño modular adaptable a distintas necesidades y la capacidad de monitoreo en tiempo real, características que los hacen ideales para aplicaciones de control y automatización industrial.

El microcontrolador ESP32

El ESP32 es un microcontrolador de bajo costo desarrollado por Espressif Systems, que integra conectividad Wi-Fi y Bluetooth de doble modo. Cuenta con un procesador Xtensa de arquitectura de 32 bits (disponible en versiones de uno o dos núcleos) capaz de operar hasta 240 MHz, además de memoria SRAM y ROM integradas.

Entre sus características más relevantes para el desarrollo de proyectos de control se encuentran:

- Hasta 34 pines GPIO (General Purpose Input/Output) configurables como entradas o salidas digitales.
- Dos convertidores analógico-digitales (ADC) de 12 bits, con hasta 18 canales.
- Dos convertidores digital-analógicos (DAC) de 8 bits.
- Múltiples interfaces de comunicación: SPI, I2C, UART e I2S.
- Módulo de PWM con hasta 16 canales, útil para el control de motores y LEDs.

Gracias a la multiplexación de pines, la mayoría de las funciones del ESP32 pueden reasignarse a diferentes GPIO según las necesidades del proyecto. No obstante, existen pines de propósito especial —como los pines de arranque (strapping) o los de entrada únicamente (GPIO 34 a 39)— que deben evitarse o utilizarse con precaución, ya que intervienen en la configuración del microcontrolador durante el encendido.

Entradas y salidas digitales

Los microcontroladores interpretan el mundo físico a través de sus terminales de entrada y salida. Una entrada permite que el microcontrolador reciba una señal de voltaje y, a partir de ella, determine si un evento ha ocurrido (por ejemplo, si un botón fue presionado). Una salida, en cambio, es un terminal mediante el cual el microcontrolador entrega una señal de voltaje hacia un componente externo, como un LED. Cuando estas señales solo pueden tomar dos valores discretos, se les denomina señales digitales, en contraposición a las señales analógicas, cuyo voltaje varía de forma continua en función de la magnitud que representan. En el caso de una entrada digital, el microcontrolador únicamente distingue dos estados: HIGH (nivel lógico alto) y LOW (nivel lógico bajo), sin importar variaciones intermedias de voltaje.

Niveles lógicos

Un nivel lógico es la tensión específica o el estado en el que puede encontrarse una señal digital. En electrónica digital se manejan dos estados: encendido (ON) y apagado (OFF), representados en binario como 1 y 0 respectivamente. El estándar más utilizado para definir estos niveles es la lógica TTL (Transistor-Transistor Logic), la cual establece rangos de voltaje aceptables tanto para la entrada como para la salida de un dispositivo digital.

Para un sistema TTL estándar de 5 V se definen los siguientes umbrales:

- **VIH** (tensión mínima de entrada para un nivel alto): aproximadamente 2 V.
- **VIL** (tensión máxima de entrada para un nivel bajo): aproximadamente 0.8 V.
- **VOH** (tensión mínima de salida para un nivel alto): aproximadamente 2.7 V.
- **VOL** (tensión máxima de salida para un nivel bajo): aproximadamente 0.5 V.

La diferencia entre los rangos tolerables de entrada y salida se conoce como margen de ruido, y representa la cantidad de tensión indeseada que un sistema puede tolerar sin interpretar erróneamente una señal. Este concepto es relevante al diseñar circuitos digitales, ya que garantiza que los niveles HIGH y LOW se reconozcan de forma confiable incluso ante pequeñas variaciones de voltaje.

Operación lógica AND

La operación AND requiere que todas las condiciones de entrada sean verdaderas para activar la salida.

Para dos entradas A y B: $\boldsymbol{Y = A * B}$ $\boldsymbol{(1)}$

Su tabla de verdad es:

Tabla 1. Tabla de verdad de la función AND

Entrada A	Entrada B	Salida Y
0	0	0
0	1	0
1	0	0
1	1	1

Operación lógica OR

La operación OR, a diferencia de la AND, activa la salida cuando al menos una de las entradas es verdadera.

Para dos entradas A y B: $\boldsymbol{Y = A + B}$ $\boldsymbol{(2)}$

Su tabla de verdad es:

Tabla 2. Tabla de verdad de la función OR

Entrada A	Entrada B	Salida Y
0	0	0
0	1	1
1	0	1
1	1	1

Control eléctrico y su aplicación en la automatización

El control eléctrico consiste en el uso de dispositivos eléctricos y electrónicos para regular, monitorear y automatizar el funcionamiento de sistemas y máquinas. Puede clasificarse en control manual, cuando requiere la intervención directa de un operador, o control automático, cuando se apoya en sensores, relés y controladores programables (PLC) que ajustan el comportamiento del sistema sin intervención humana constante. Este principio es la base conceptual de la lógica digital aplicada en la presente práctica, donde las entradas (botones) y la salida (LED) simulan, a pequeña escala, el comportamiento de un sistema de control automatizado.

Sensor de temperatura digital DS18B20

El DS18B20 es un sensor digital de temperatura que mide de −55 a 125 °C con una precisión de ±0.5 °C. Su resolución se puede seleccionar entre 9 y 12 bits, y su salida es un bus 1-Wire que se conecta a un pin digital del microcontrolador. [9] El sensor funciona con una alimentación de 3.0 a 5.5 V y puede tardar hasta 750 ms en convertir la temperatura cuando trabaja a 12 bits de resolución. [10] Como entrega el dato ya digitalizado, no requiere un convertidor analógico-digital, y se programa con las librerías `OneWire` y `DallasTemperature`.

Control ON-OFF con histéresis

En el control ON-OFF o de dos posiciones, el actuador solo se enciende o se apaga según si la variable está por encima o por debajo de la referencia. [12] Para evitar que la salida conmute continuamente por cambios mínimos de temperatura, lo que acorta la vida del relevador, se crea una banda de histéresis entre los puntos de encendido y apagado. [11]

En esta práctica los umbrales LL y HL definen esa banda. El sistema actúa según la temperatura medida T:

$$\begin{cases} T \leq LL: foco\ ON, Peltier\ OFF \\ T \geq HL: foco\ OFF, Peltier\ ON \\ LL < T < HL: se\ conserva\ en\ el\ ultimo\ estado \end{cases}$$

Además, la temperatura se muestra en la pantalla como porcentaje de un rango de 30 a 60 °C:

$$P = \frac{T - 30}{60 - 30} * 100$$

Un sistema térmico responde con retraso, pues su temperatura tarda en reaccionar a la acción del calefactor o del enfriador. Por eso el control ON-OFF mantiene la temperatura dentro de un rango, no en un valor exacto.

Calentamiento con foco y enfriamiento con celda Peltier

El foco funciona como calefactor, ya que gran parte de la energía eléctrica que consume se libera en forma de calor. Para enfriar se utiliza una celda Peltier. El enfriamiento termoeléctrico se basa en el efecto Peltier, que genera un flujo de calor entre la unión de dos materiales distintos. El lado caliente se une a un disipador para mantenerse cerca de la temperatura ambiente, mientras el lado frío baja por debajo de ella. [13] Al aplicar voltaje de corriente directa, el módulo absorbe calor de una superficie y lo expulsa por la opuesta. El disipador debe eliminar tanto el calor extraído como el que genera el propio paso de la corriente. [14] Por eso la celda trabaja siempre acompañada de su disipador.

Relevador

Un relevador es un interruptor automático en el que una corriente pequeña controla una corriente mayor, y se usa para evitar conectar un interruptor directamente a un circuito de corriente alterna, lo que representaría un riesgo para las personas. El módulo tiene terminales NO (normalmente abierto) y NC (normalmente cerrado). [15] En esta práctica tres relevadores controlan el foco, la celda Peltier y el ventilador de circulación, de modo que el ESP32 maneja únicamente las señales de control.

Bus I2C y pantalla LCD

El ESP32 tiene dos interfaces I2C que operan como maestro o esclavo. Sus pines predeterminados son GPIO 21 (SDA) y GPIO 22 (SCL), y cada dispositivo del bus tiene una dirección propia. [16] La LCD I2C combina un display HD44780 con un adaptador que convierte los datos del bus I2C en información paralela, y se conecta con solo cuatro pines. [17] En esta práctica la pantalla usa la dirección 0x27 y los pines predeterminados 21 y 22.

Control ON-OFF de temperatura

PRÁCTICA 3. Sensor de temperatura y foco

Objetivo

Implementar un sistema de control de temperatura de tipo ON-OFF con histéresis, utilizando una tarjeta ESP32, un sensor digital DS18B20 para medir la temperatura dentro de una caja y una pantalla LCD I2C para visualizarla. A partir de dos umbrales configurables por el usuario (nivel bajo LL y nivel alto HL), el sistema debe activar un foco como elemento calefactor cuando la temperatura baje del límite inferior y una celda Peltier con disipador cuando supere el límite superior, manteniendo la temperatura dentro del rango deseado. Con ello se aplican los conceptos de sensores digitales, control por histéresis, comunicación I2C, antirrebote de botones y control de cargas mediante relevadores.

Material requerido

- 1 tarjeta ESP32.
- 1 sensor de temperatura DS18B20.
- 1 resistencia de 4.7 kΩ (pull-up del sensor).
- 1 pantalla LCD 16x2 con módulo adaptador I2C.
- 3 botones pulsadores (SEL, UP, DOWN).
- 3 módulos relevadores de 1 canal.
- 1 foco (calentador) con su socket.
- 1 celda Peltier con disipador.
- 1 ventilador de circulación.
- Fuente de alimentación externa para las cargas.
- Caja de cartón (cámara de pruebas).
- Protoboard.
- Cables Dupont.
- Cable USB.
- Computadora con Arduino IDE.

Asignación de pines

Componente	Pin ESP32	Tipo	Función
Sensor DS18B20 (DQ)	GPIO 4	Entrada/salida digital (1-Wire)	Lectura de temperatura
LCD I2C (SDA)	GPIO 21	I2C (datos)	Comunicación con la pantalla

LCD I2C (SCL)	GPIO 22	I2C (reloj)	Comunicación con la pantalla
Botón UP	GPIO 12	Entrada digital (INPUT_PULLUP)	Incrementa el valor en edición (+5 °C)
Botón DOWN	GPIO 13	Entrada digital (INPUT_PULLUP)	Decrementa el valor en edición (−5 °C)
Botón SEL	GPIO 14	Entrada digital (INPUT_PULLUP)	Cambia entre modo normal, edición de LL y edición de HL
Relevador del foco	GPIO 18	Salida digital	Enciende/apaga el foco (calentador)
Relevador de la celda Peltier	GPIO 19	Salida digital	Enciende/apaga la Peltier con disipador (enfriamiento)
Relevador del ventilador	GPIO 23	Salida digital	Mantiene encendido el ventilador de circulación de aire

Desarrollo

Diseño del circuito

El sistema mide la temperatura interna de una caja de cartón mediante el sensor **DS18B20**, un sensor digital que se comunica por el protocolo 1-Wire utilizando un solo pin de datos (GPIO 4). Los datos viajan ya digitalizados, por lo que no se requiere conversión analógica-digital, y el sensor necesita una resistencia pull-up de 4.7 kΩ entre su línea de datos y la alimentación.

La temperatura se muestra en una **LCD 16x2 I2C** (GPIO 21 y 22), junto con un porcentaje de temperatura (0% equivale a 30 °C y 100% a 60 °C) y los umbrales LL y HL vigentes. Tres botones permiten configurar dichos umbrales: SEL alterna entre modo normal, edición de LL y edición de HL (el cursor parpadea sobre el valor que se edita), mientras que UP y DOWN lo modifican en pasos de 5 °C, dentro de un rango de 30 a 60 °C.

Para actuar sobre la temperatura se emplean **tres relevadores**: uno controla el foco, que funciona como calentador; otro controla la celda Peltier con su disipador, que enfría; y un tercero mantiene encendido de forma permanente un ventilador de circulación, para distribuir el aire dentro de la caja y lograr una lectura más uniforme. Los relevadores aíslan el circuito de control (ESP32) de las cargas de mayor voltaje y corriente.

El control implementado es de tipo **ON-OFF con histéresis**: no se busca mantener una temperatura exacta, sino dentro de una banda entre LL y HL. Cuando la temperatura baja hasta LL o menos, se enciende el foco y se apaga la Peltier; cuando sube hasta HL o más, se apaga el foco y se enciende la Peltier. Entre ambos umbrales el sistema conserva el último estado activo, lo que evita encendidos y apagados continuos de las cargas.

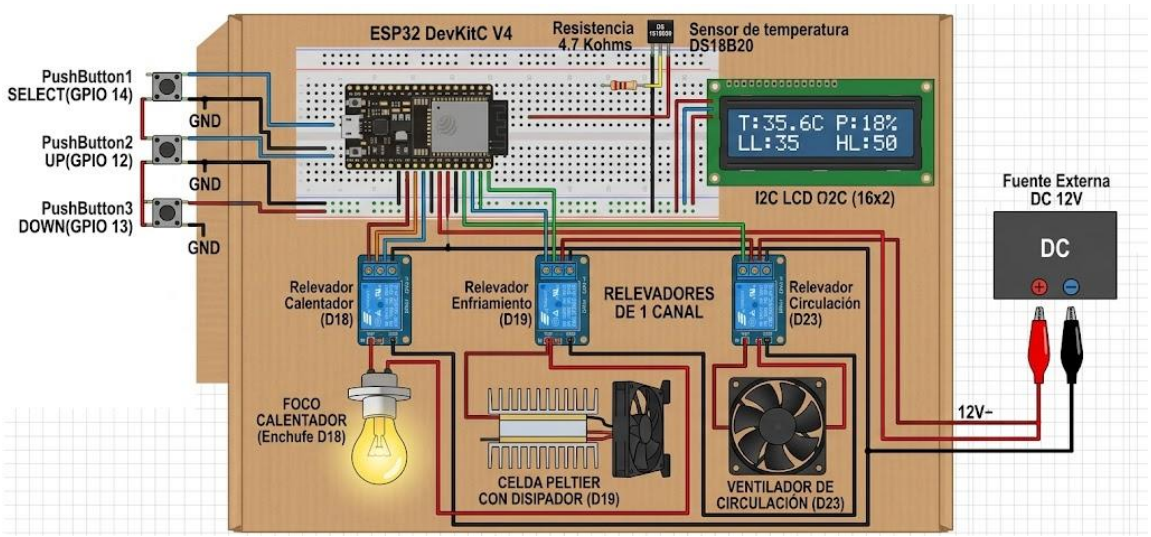


Figura 1. Circuito de la Practica Control ON-OFF de temperatura

Conexiones

- **DS18B20 (DQ) → GPIO 4**, con resistencia de 4.7 kΩ entre DQ y VCC; VCC y GND del sensor a la alimentación y tierra de la tarjeta.
- **GPIO 21 (SDA) / GPIO 22 (SCL) → Módulo I2C de la LCD**, con alimentación y GND correspondientes.
- **GPIO 14 → Botón SEL → GND:** selecciona el modo de edición.
- **GPIO 12 → Botón UP → GND:** incrementa el valor seleccionado.
- **GPIO 13 → Botón DOWN → GND:** decrementa el valor seleccionado.
- **GPIO 18 → Pin IN del relevador del foco:** controla el calentamiento.
- **GPIO 19 → Pin IN del relevador de la Peltier:** controla el enfriamiento.
- **GPIO 23 → Pin IN del relevador del ventilador:** mantiene la circulación de aire.
- **VCC y GND de los relevadores → alimentación y GND del ESP32**, con GND común entre todos los circuitos.
- **Contactos de potencia de los relevadores → foco, Peltier y ventilador**, con su respectiva fuente de alimentación, aislados del circuito de control.

Programación

Código en Wokwi primera parte del circuito (estructura)

```
{
  "version": 1,
  "author": "Sistemas Embebidos",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left":
0, "attrs": {} },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd",
      "top": 102.4,
      "left": 341.6,
      "attrs": { "pins": "i2c", "i2cAddress": "0x27" }
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": 121.2,
      "left": 138.6,
      "attrs": { "color": "red", "flip": "1" }
    },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": 169.2,
      "left": 138.6,
      "attrs": { "color": "red", "flip": "1" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": 157.55,
      "left": 172.8,
      "attrs": { "value": "1000" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": 205.55,
      "left": 172.8,
      "attrs": { "value": "1000" }
    },
    {
      "type": "wokwi-pushbutton-6mm",
      "id": "btn1",
      "top": 36.2,
      "left": -115.2,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton-6mm",
      "id": "btn2",
      "top": 74.6,
      "left": -115.2,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
```

```
      "type": "wokwi-pushbutton-6mm",
      "id": "btn3",
      "top": 113,
      "left": -115.2,
      "attrs": { "color": "green", "xray": "1" }
    },
    { "type": "wokwi-gnd", "id": "gnd1", "top": 163.2, "left": -154.2,
"attrs": {} },
    { "type": "wokwi-ds18b20", "id": "temp1", "top": -106.73, "left":
176.88, "attrs": {} },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -34.45,
      "left": 48,
      "attrs": { "value": "4700" }
    }
  ],
  "connections": [
    [ "esp:21", "lcd:SDA", "green", [ "h216.04", "v76.6" ] ],
    [ "esp:22", "lcd:SCL", "blue", [ "h225.64", "v114.9" ] ],
    [ "esp:5V", "lcd:VCC", "red", [ "h-23.81", "v48", "h297.6", "v-105.7"
] ],
    [ "esp:17", "led1:A", "green", [ "h33.64", "v38.4" ] ],
    [ "esp:16", "led2:A", "green", [ "h24.04", "v76.8" ] ],
    [ "r1:1", "led1:C", "green", [ "v0" ] ],
    [ "r2:1", "led2:C", "green", [ "v0" ] ],
    [ "esp:34", "btn1:2.r", "green", [ "h-14.21", "v-9.6" ] ],
    [ "esp:32", "btn2:2.r", "green", [ "h-14.21", "v0", "h-67.2", "v9.6"
] ],
    [ "esp:33", "btn3:2.r", "green", [ "h-71.81", "v38.4" ] ],
    [ "gnd1:GND", "btn3:1.l", "black", [ "v0" ] ],
    [ "btn1:1.l", "gnd1:GND", "black", [ "h0" ] ],
    [ "btn2:1.l", "gnd1:GND", "black", [ "h-28.8" ] ],
    [ "esp:4", "temp1:DQ", "yellow", [ "h43.24", "v-115.2", "h48" ] ],
    [ "esp:3V3", "temp1:VCC", "red", [ "h-23.81", "v-38.4", "h220.8" ]
],
    [ "esp:GND.2", "temp1:GND", "black", [ "v-19.2", "h81.64" ] ],
    [ "esp:CMD", "r2:2", "black", [ "h-14.21", "v48", "h259.2", "v-28.8"
] ],
    [ "r1:2", "esp:CMD", "black", [ "v0", "h18", "v76.8", "h-259.2", "v-
48" ] ],
    [ "r3:1", "esp:3V3", "red", [ "v0", "h-67.2", "v57.6" ] ],
    [ "r3:2", "temp1:DQ", "yellow", [ "v0", "h85.2" ] ]
  ],
  "dependencies": {}
}
```

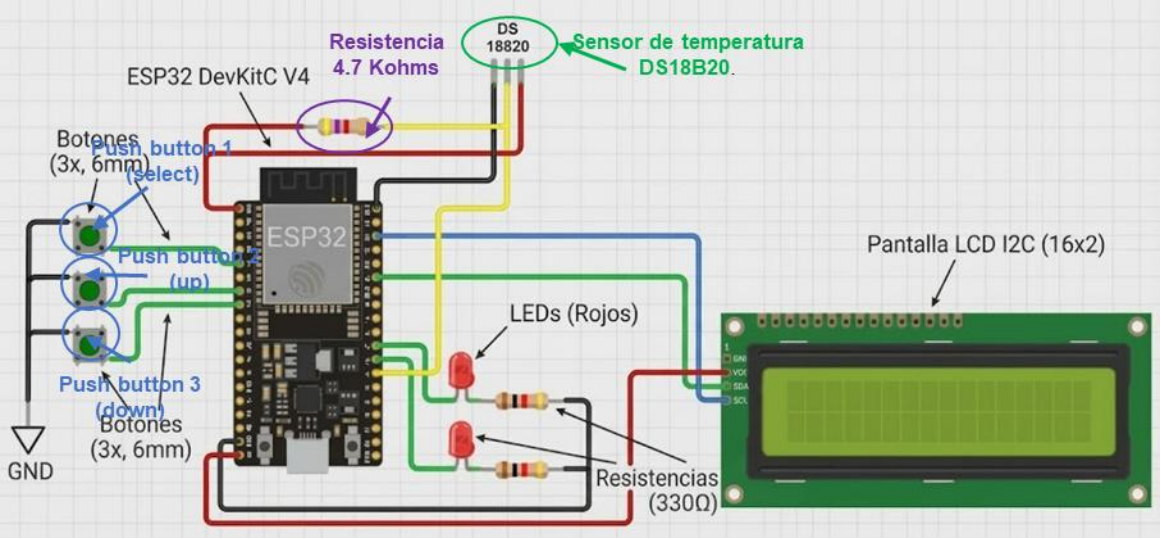


Figura 2. Primera parte del circuito en diseñado en Wokwi (estructura)

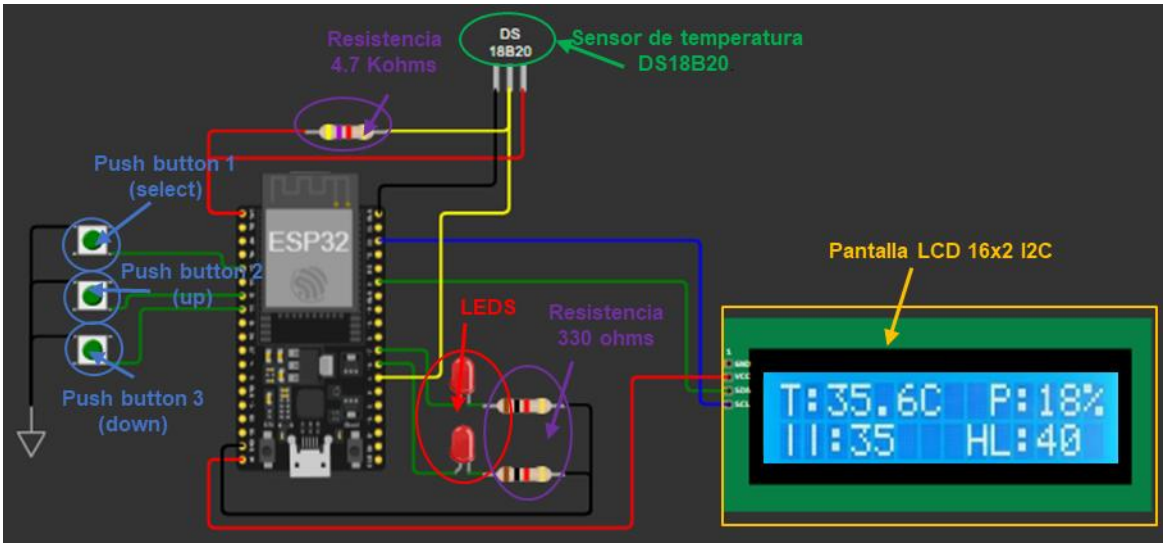


Figura 3. Primera parte del circuito en diseñado en Wokwi (en funcionamiento)

Codigo usado en ESP32 codigo Final

```
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LiquidCrystal_I2C.h>

// ========================================
// CONFIGURACIÓN DS18B20
// ========================================
```

```
#define DS18B20_PIN 4
OneWire oneWire(DS18B20_PIN);
DallasTemperature sensors(&oneWire);

// ==========================================
// CONFIGURACIÓN LCD I2C
// ==========================================
#define SDA_PIN 21
#define SCL_PIN 22
#define LCD_ADDRESS 0x27
LiquidCrystal_I2C lcd(LCD_ADDRESS, 16, 2);

// ==========================================
// PINES DE BOTONES
// ==========================================
#define PIN_BTN_UP 12
#define PIN_BTN_DOWN 13
#define PIN_BTN_SEL 14

// ==========================================
// PINES DE RELÉ
// ==========================================
#define PIN_RELE_FOCO 18        // Foco / Calentador en D18
#define PIN_RELE_PELTIER 19     // Celda Peltier + Disipador en D19
#define PIN_RELE_CIRCULACION 23 // Ventilador de circulación en D23

// Lógica de los relés (Cambiar a LOW si tus módulos se activan en bajo)
#define RELE_ON HIGH
#define RELE_OFF LOW

// ==========================================
// VARIABLES GLOBALES
// ==========================================
int ll = 35; // Low Level (Mínimo 30)
int hl = 50; // High Level (Máximo 60)
int modo_edicion = 0; // 0 = Normal, 1 = Editar LL, 2 = Editar HL

// Variables para estado de botones y anti-rebote (Debounce)
bool lastSelState = HIGH;
bool lastUpState = HIGH;
bool lastDownState = HIGH;

unsigned long lastDebounceTimeSel = 0;
unsigned long lastDebounceTimeUp = 0;
unsigned long lastDebounceTimeDown = 0;
const unsigned long debounceDelay = 200; // Tiempo de espera anti-rebote
en ms

unsigned long previousMillis = 0;
const long updateInterval = 500; // Refrescar pantalla y sensores cada
500 ms

void setup() {
    Serial.begin(115200);
    Wire.begin(SDA_PIN, SCL_PIN);
    sensors.begin();

    lcd.init();
    lcd.backlight();

    // Configuración de botones con Pull-Up interno
    pinMode(PIN_BTN_UP, INPUT_PULLUP);
```

```
    pinMode(PIN_BTN_DOWN, INPUT_PULLUP);
    pinMode(PIN_BTN_SEL, INPUT_PULLUP);

    // Configuración de salidas de los relés
    pinMode(PIN_RELE_FOCO, OUTPUT);
    pinMode(PIN_RELE_PELTIER, OUTPUT);
    pinMode(PIN_RELE_CIRCULACION, OUTPUT);

    // Estado inicial: Foco y Peltier apagados, Circulación SIEMPRE
encendido
    digitalWrite(PIN_RELE_FOCO, RELE_OFF);
    digitalWrite(PIN_RELE_PELTIER, RELE_OFF);
    digitalWrite(PIN_RELE_CIRCULACION, RELE_ON);

    // Mensaje inicial
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sistema Control");
    lcd.setCursor(0, 1);
    lcd.print("Iniciando...");
    delay(2000);
    lcd.clear();
}

void loop() {
    unsigned long currentMillis = millis();

    // -----------------------------------------------------
    // 1. LECTURA Y LÓGICA DE BOTÓN SELECCIÓN (D14)
    // -----------------------------------------------------
    bool readingSel = digitalRead(PIN_BTN_SEL);

    // Detectar pulsación con filtro anti-rebote
    if (readingSel == LOW && lastSelState == HIGH && (currentMillis -
lastDebounceTimeSel > debounceDelay)) {
        lastDebounceTimeSel = currentMillis;
        modo_edicion++;
        if (modo_edicion > 2) modo_edicion = 0;

        if (modo_edicion == 0) {
            lcd.noBlink(); // Apagar parpadeo del cursor
        } else {
            lcd.blink();   // Activar parpadeo del cursor
        }

        // Sincronizar lectura de los otros botones para evitar falsos
disparos al cambiar de modo
        lastUpState = digitalRead(PIN_BTN_UP);
        lastDownState = digitalRead(PIN_BTN_DOWN);
        lastDebounceTimeUp = currentMillis;
        lastDebounceTimeDown = currentMillis;
    }
    lastSelState = readingSel;

    // -----------------------------------------------------
    // 2. LECTURA DE BOTONES SUBIR (D12) Y BAJAR (D13)
    // -----------------------------------------------------
    if (modo_edicion != 0) {
        // Botón SUBIR (D12)
        bool readingUp = digitalRead(PIN_BTN_UP);
        if (readingUp == LOW && lastUpState == HIGH && (currentMillis -
lastDebounceTimeUp > debounceDelay)) {
```

```
            lastDebounceTimeUp = currentMillis;
            if (modo_edicion == 1 && ll < 60) ll += 5;
            if (modo_edicion == 2 && hl < 60) hl += 5;

            previousMillis = currentMillis - updateInterval; // 
Refrescar pantalla LCD de inmediato
        }
        lastUpState = readingUp;

        // Botón BAJAR (D13)
        bool readingDown = digitalRead(PIN_BTN_DOWN);
        if (readingDown == LOW && lastDownState == HIGH && (currentMillis
- lastDebounceTimeDown > debounceDelay)) {
            lastDebounceTimeDown = currentMillis;
            if (modo_edicion == 1 && ll > 30) ll -= 5;
            if (modo_edicion == 2 && hl > 30) hl -= 5;

            previousMillis = currentMillis - updateInterval; // 
Refrescar pantalla LCD de inmediato
        }
        lastDownState = readingDown;
    } else {
        // Fuera de modo edición, mantener actualizados los estados de 
referencia
        lastUpState = digitalRead(PIN_BTN_UP);
        lastDownState = digitalRead(PIN_BTN_DOWN);
    }

    // ------------------------------------------------------
    // 3. CONTROL DE SENSORES, RELÉS Y PANTALLA LCD
    // ------------------------------------------------------
    if (currentMillis - previousMillis >= updateInterval) {
        previousMillis = currentMillis;

        // Solicitar medición
        sensors.requestTemperatures();
        float temperatura = sensors.getTempCByIndex(0);

        // Control por Histéresis
        if (temperatura != DEVICE_DISCONNECTED_C) {
            if (temperatura <= ll) {
                // Al llegar o bajar de LL: Enciende FOCO, apaga PELTIER
                digitalWrite(PIN_RELE_FOCO, RELE_ON);
                digitalWrite(PIN_RELE_PELTIER, RELE_OFF);
            }
            else if (temperatura >= hl) {
                // Al llegar o subir de HL: Apaga FOCO, enciende PELTIER
                digitalWrite(PIN_RELE_FOCO, RELE_OFF);
                digitalWrite(PIN_RELE_PELTIER, RELE_ON);
            }
            // Entre LL y HL conserva el último estado activo
        }

        // Cálculo de porcentaje (0% = 30°C | 100% = 60°C)
        int porcentaje = 0;
        if (temperatura <= 30.0) {
            porcentaje = 0;
        } else if (temperatura >= 60.0) {
            porcentaje = 100;
        } else {
            porcentaje = (int)(((temperatura - 30.0) / 30.0) * 100.0);
        }
```

```
        // Fila 1: T:XX.XC P:XX%
        lcd.setCursor(0, 0);
        lcd.print("T:");
        if (temperatura == DEVICE_DISCONNECTED_C) {
            lcd.print("Error ");
        } else {
            lcd.print(temperatura, 1);
            lcd.print("C ");
        }
        lcd.print("P:");
        lcd.print(porcentaje);
        lcd.print("%   ");

        // Fila 2: LL:XX  HL:XX
        lcd.setCursor(0, 1);
        lcd.print("LL:");
        lcd.print(ll);
        lcd.print("  HL:");
        lcd.print(hl);
        lcd.print("   ");

        // Posicionar el cursor para el parpadeo en la LCD
        if (modo_edicion == 1) {
            lcd.setCursor(3, 1);  // Parpadea sobre el valor de LL
        } else if (modo_edicion == 2) {
            lcd.setCursor(10, 1); // Parpadea sobre el valor de HL
        }
    }
}
```

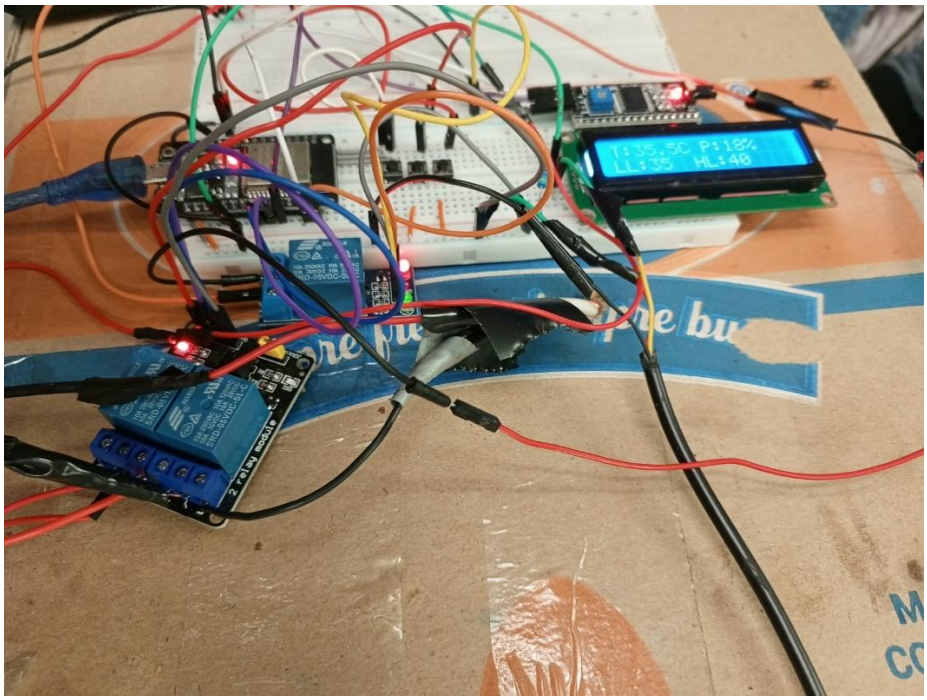


Figura 4. Vista General del Circuito Físico

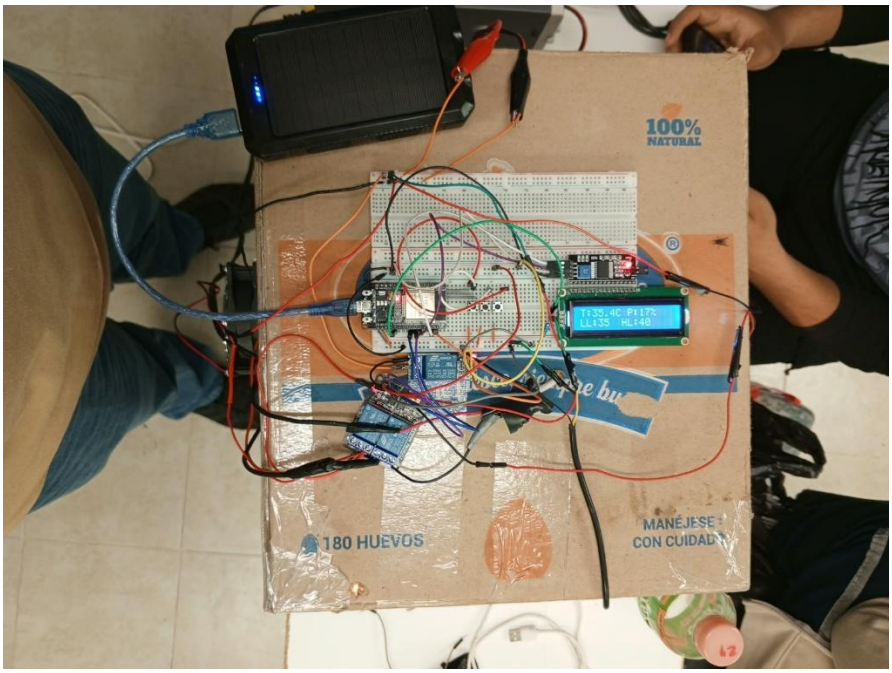


Figura 5. Vista General del Circuito Físico en Funcionamiento



Figura 6. Ventilador Implementado en la Caja

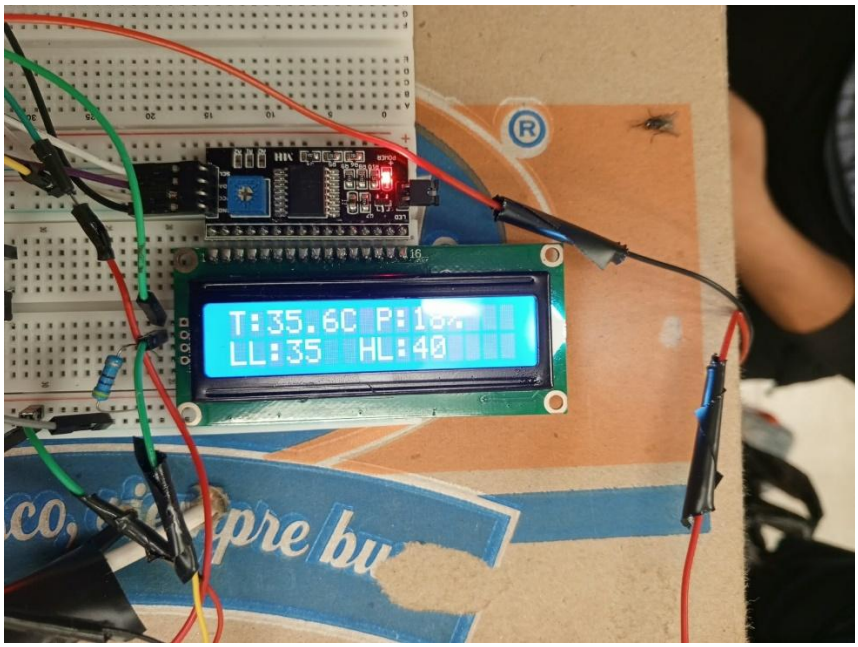


Figura 7. Datos Obtenidos Durante el Funcionamiento de la Practica

- **Configuración de periféricos:** se definen el sensor DS18B20 (GPIO 4), la LCD I2C (dirección 0x27, pines 21 y 22), los botones con INPUT_PULLUP y los relevadores como salidas. Los relevadores se activan en nivel alto (`RELE_ON = HIGH`).
- **Estado inicial:** en `setup()` el foco y la Peltier inician apagados y el ventilador de circulación queda siempre encendido.
- **Antirrebote por software:** cada botón compara su estado actual con el anterior y verifica que hayan pasado al menos 200 ms desde la última pulsación válida, evitando lecturas dobles.
- **Modos de edición:** la variable modo_edicion toma tres valores (0 normal, 1 edita LL, 2 edita HL), y con `lcd.blink`() se indica en pantalla cuál parámetro se está modificando.
- **Temporización sin bloqueo:** en lugar de `delay()`, se usa `millis()` para refrescar sensores, relevadores y pantalla cada 500 ms, de modo que los botones responden sin retrasos.
- **Control por histéresis:** si `temperatura <= ll` se enciende el foco y se apaga la Peltier; si `temperatura >= hl` ocurre lo contrario; entre ambos límites se conserva el estado anterior.
- **Manejo de errores:** si el sensor se desconecta (`DEVICE_DISCONNECTED_C`), el programa no modifica los relevadores y muestra “Error” en pantalla.

- **Visualización:** la fila 1 muestra la temperatura y su porcentaje (`T:XX.XC P:XX%`) y la fila 2 los umbrales (`LL:XX HL:XX`).

Funcionamiento

1. Al energizar el sistema, se inicializan la comunicación serial, el bus I2C, el sensor y la LCD. El foco y la Peltier quedan apagados, el ventilador de circulación se enciende y la pantalla muestra “Sistema Control / Iniciando...” durante 2 segundos.
2. El sistema inicia con los umbrales por defecto LL = 35 °C y HL = 50 °C.
3. Cada 500 ms se solicita la temperatura al DS18B20 y se actualiza la pantalla con la temperatura, el porcentaje y los umbrales.
4. Si la temperatura es menor o igual a LL, se activa el relevador del foco y se desactiva el de la Peltier, de modo que la caja se calienta.
5. Si la temperatura es mayor o igual a HL, se desactiva el foco y se activa la Peltier con su disipador, de modo que la caja se enfría.
6. Mientras la temperatura se encuentra entre LL y HL, el sistema mantiene el último estado activo.
7. Al presionar SEL se entra a edición de LL (cursor parpadeando sobre LL) y al presionarlo de nuevo a edición de HL; con UP y DOWN se ajusta el valor en pasos de 5 °C (límites de 30 a 60 °C), y una tercera pulsación regresa al modo normal.
8. El ciclo se repite continuamente mientras el sistema esté energizado.

Resultados

Durante las pruebas, el sensor DS18B20 entregó lecturas de temperatura estables, mostradas correctamente en la LCD junto con el porcentaje y los umbrales configurados. Al encontrarse la temperatura en LL o por debajo, el relevador del foco se activó y la caja comenzó a calentarse; al alcanzar HL, el foco se apagó y se activó la celda Peltier con su disipador para enfriar. Los botones permitieron modificar LL y HL sin lecturas dobles, gracias al filtro antirrebote.

Un resultado importante fue la **respuesta lenta del sistema térmico**: la caja de cartón tardó en calentarse (aprox. 15 - 20 *minutos]*), por lo que la temperatura subía de forma gradual y el cambio de estado de las cargas no era inmediato. Una vez superados los ajustes iniciales de conexión, el sistema operó de manera estable y mantuvo la temperatura dentro del rango definido.

Observaciones y conclusiones

Esta práctica permitió aplicar un control ON-OFF con histéresis sobre una variable física real. A diferencia de las prácticas anteriores, donde la salida respondía de inmediato a un botón, aquí la salida (calor) afecta a una variable que cambia lentamente, por lo que se observó el comportamiento propio de los sistemas térmicos: la temperatura no responde al instante cuando se enciende el foco. Esto se notó especialmente por el material de la caja, ya que el cartón es un aislante que tarda en acumular calor. Para esperar el efecto de las cargas, en lugar de forzar cambios, se mantuvo la banda de histéresis, que además protege a los relevadores de conmutaciones continuas.

Como complicación, se presentaron **pequeñas fallas de conexión** al armar el circuito, normales en montajes con muchos cables (sensor, LCD, botones y tres relevadores). Se resolvieron revisando la continuidad y firmeza de cada conexión, lo que confirmó que muchas fallas en sistemas embebidos son de origen físico y no del programa. Después de ello el sistema funcionó correctamente.

Respecto a la relación entre programa y circuito, el código implementó mejoras respecto a la Práctica 2: el **antirrebote por software** evitó los problemas de doble pulsación y el uso de `millis()` en lugar de `delay()` mantuvo la interfaz de botones receptiva mientras se leía el sensor. También se observó que cada elemento físico tiene su contraparte en el programa: cada relevador corresponde a una salida, el DS18B20 a una lectura periódica y los botones a los modos de edición.

Finalmente, se comprobó que un control ON-OFF con histéresis es suficiente para mantener una temperatura dentro de un rango, aunque no ofrece un control preciso. Para mayor exactitud podrían explorarse estrategias como el control proporcional o PID.

Referencias

[1] TRBL Services. (s. f.). *Sistema embebido y sus características: Conceptos fundamentales*. TRBL Services. https://trbl-services.eu/blog-sistema-embebido-caracteristicas/

[2] SD Industrial. (s. f.). *Sistemas embebidos*. SD Industrial. https://sdindustrial.com.mx/blog/sistemas-embebidos/

[3] Llamas, L. (s. f.). *Pinout y detalles del hardware del ESP32*. Luis Llamas. https://www.luisllamas.es/esp32-detalles-hardware-pinout/

[4] Acebott. (s. f.). *Introducción al ESP32*. Acebott. https://acebott.com/es/docs/getting-started-with-esp32/

[5] Panama Hitek. (s. f.). *Entradas y salidas analógicas y digitales*. Panama Hitek. https://panamahitek.com/entradas-y-salidas-analogicas-y-digitales/

[6] MCI Electronics. (2022). *Niveles lógicos*. MCI Electronics. https://cursos.mcielectronics.cl/2022/08/05/niveles-logicos/

[7] All About Circuits. (s. f.). *Niveles de voltaje de señales lógicas*. All About Circuits. https://www.allaboutcircuits.com/textbook/digital/chpt-3/logic-signal-voltage-levels/

[8] Datatechnic. (s. f.). *Control eléctrico: Fundamentos y aplicaciones en la industria moderna*. Datatechnic. https://datatechnic.com.mx/control-electrico-conceptos-basicos-y-aplicaciones-en-la-industria/

[9] SunFounder. (s. f.). *Módulo sensor de temperatura (DS18B20)*. SunFounder. https://docs.sunfounder.com/projects/umsk/es/latest/01_components_basic/18-component_ds18b20.html

[10] OpenELAB. (s. f.). *Sensor digital de temperatura DS18B20 | Termómetro 1-Wire*. OpenELAB. https://openelab.io/es/products/ds18b20digitaltemperaturesensor

[11] OMRON. (s. f.). *What is "hysteresis"?* OMRON Industrial Automation. https://www.ia.omron.com/support/faq/answer/53/faq00549/

[12] Llamas, L. (s. f.). *On/Off control with hysteresis*. Luis Llamas. https://www.luisllamas.es/en/on-off-control-with-hysteresis/

[13] Profesional Review. (2018). *¿Qué es la célula Peltier y cómo funciona?* Profesional Review. https://www.profesionalreview.com/2018/10/16/que-celula-peltier/

[14] DigiKey. (s. f.). *Cómo integrar un módulo Peltier en un sistema de gestión térmica*. DigiKey. https://www.digikey.com/es/articles/how-to-integrate-a-peltier-module-into-a-thermal-management-system

[15] Keyestudio. (2024). *How to use relay with ESP32*. Keyestudio. https://keyestudio.com/blog/how-to-use-relay-with-esp32-241

[16] Random Nerd Tutorials. (s. f.). *ESP32 I2C communication: Set pins, multiple bus interfaces and peripherals (Arduino IDE)*. Random Nerd Tutorials. https://randomnerdtutorials.com/?p=89386

[17] Last Minute Engineers. (s. f.). *How to use an I2C LCD display with ESP32*. Last Minute Engineers. https://lastminuteengineers.com/esp32-i2c-lcd-tutorial/